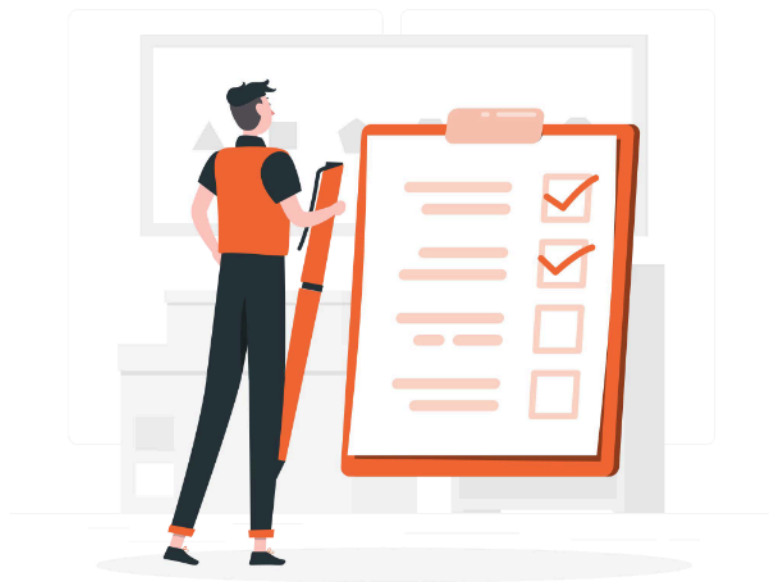


RAG Architecture & Retrieval Strategies



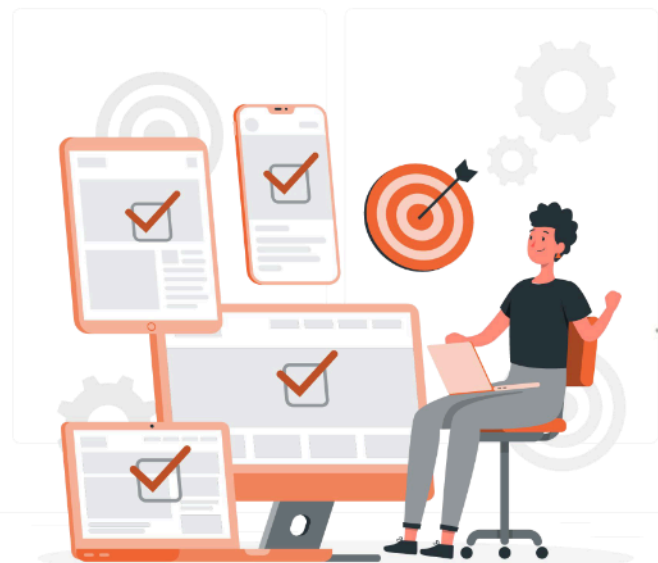
Today's Agenda

1. RAG Architecture Patterns
2. Retrieval Strategies
3. Grounding & Context Control
4. Query Rewriting & Expansion
5. Failure Modes in RAG



Learning Outcome

1. Identify fundamental RAG architecture
2. Understand basic retrieval strategies
3. Utilize grounding & context control
4. Implement query rewriting & expansion
5. Identify failure modes in RAG





RAG Architecture Patterns

GPT-5.2 Pro is our smartest and most trustworthy model yet for difficult questions where a higher-quality answer is worth the wait. In early testing, it shows fewer major errors and stronger performance across complex domains like programming.

All three models (Instant, Thinking, and Pro) have a new **knowledge cutoff of August 2025**. For users, this means GPT-5.2 starts with a more current understanding of the world, so answers are more accurate and useful, with more relevant examples and context, even before turning to web search.

Read our [blog](#) for more details on examples and evals.

[Chatgpt - Release Notes](#)

RAG Architecture

LLM Knowledge Cutoffs & Hallucinations

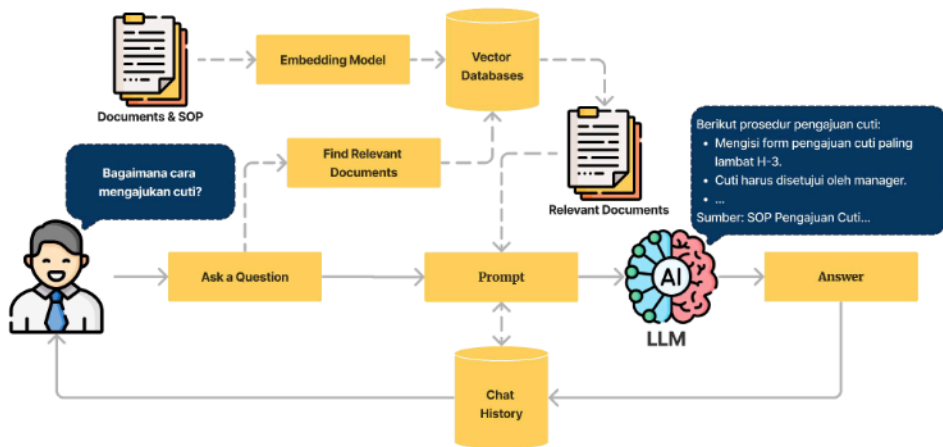
LLMs are **trained on data up to a fixed date**. They have zero awareness of anything that happened after training. When the **model doesn't know, it invents the information confidently**.

RAG Architecture

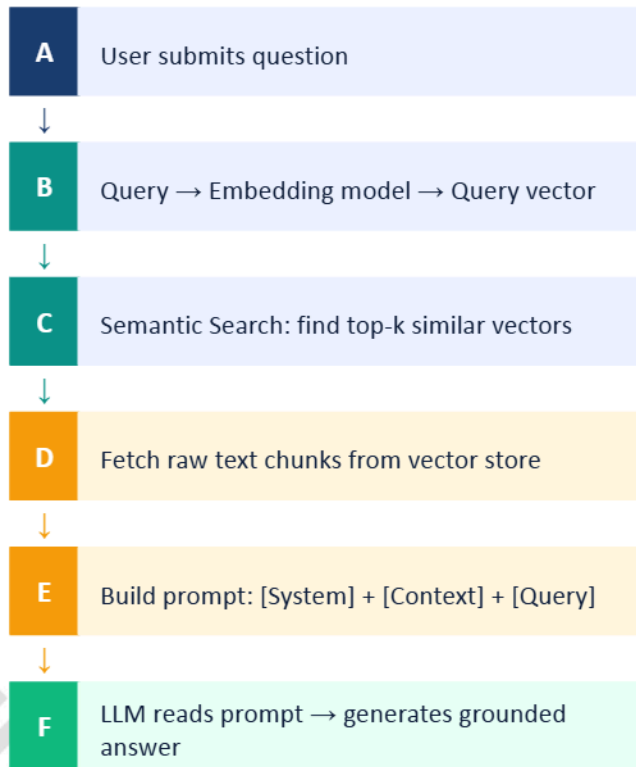
Retrieval Augmented Generation

Technique to **provide additional relevant information** for model to create more accurate answers.

- Knowledge storing
- Retrieval
- Generating responses



Simple RAG

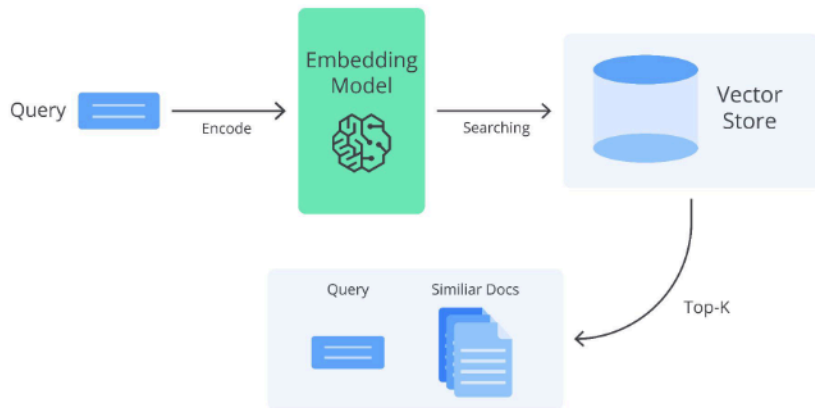


Prompt Anatomy

[SYSTEM]	You are an expert assistant. Answer ONLY using the provided context. If unsure, say 'I don't know'.
[CONTEXT]	— Document A, p.4 — "The quarterly revenue grew 23% driven by cloud services..." — Document B, p.11 — "Enterprise subscriptions rose to 12,000 accounts..."
[QUESTION]	What drove Q3 revenue growth?
[ANSWER]	According to Document A, Q3 revenue grew 23% primarily from cloud services. Document B notes enterprise subscriptions also rose significantly.



Retrieval Strategies

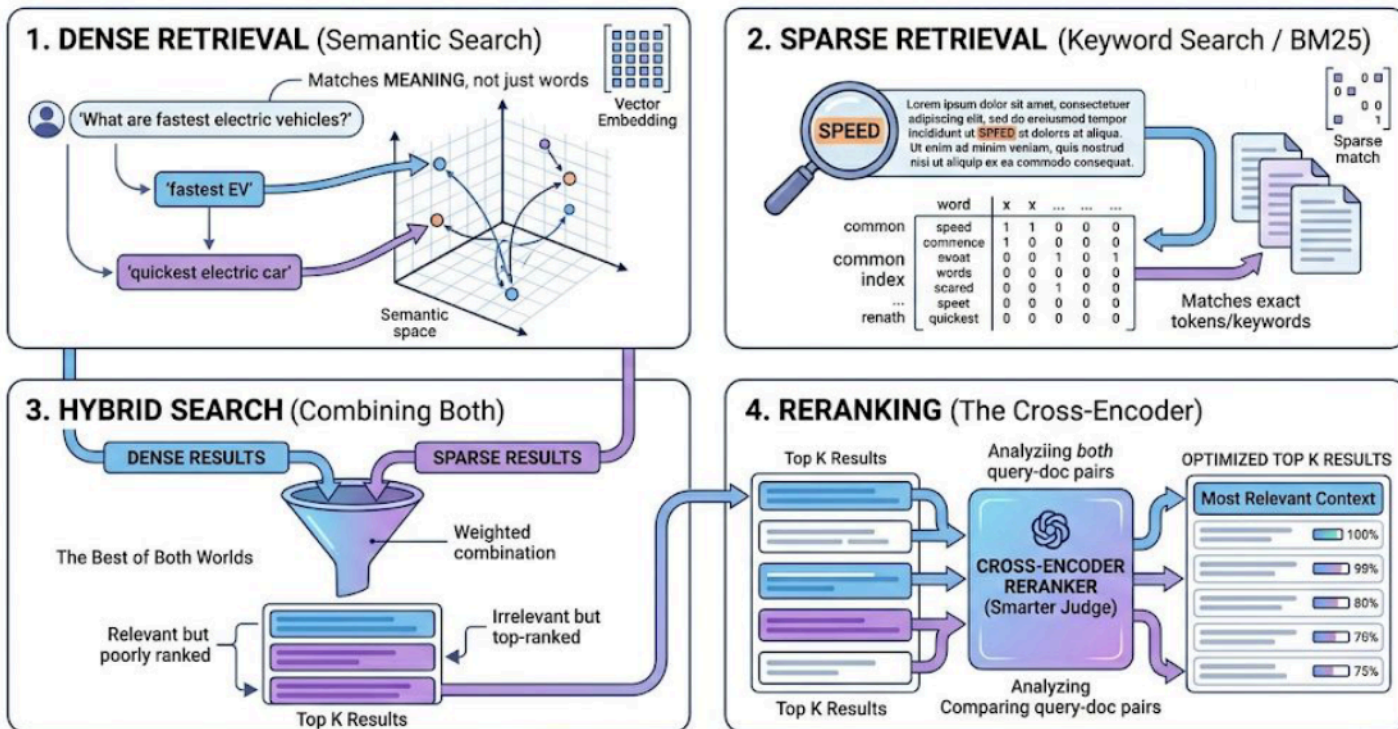


Retrieval Strategies

Retrieval: Find Relevant Information

The **process of fetching relevant information** from knowledge base to enhance LLM responses. It involves **transforming user queries into vectors, searching** a curated knowledge (from vector database) based on **semantic matches**.

Retrieval Strategies



Comparing Retrieval Strategies

Retrieval Technique	Best for/When to Use	Pros (+)	Cons (-)
1. Sparse Retrieval (Keyword Search / BM25)	• Finding precise named entities (e.g., product IDs, names, codes). • Initial filtering based on exact matches.	• Very fast and lightweight. • Highly accurate for exact terms. • Transparent and easy to implement.	• Misses conceptual context and synonyms (low semantic recall). • Sensitive to typos. • Fails on "vibes" or abstract queries.
2. Dense Retrieval (Semantic Search / Vector)	• Finding conceptually related answers, not just matching words. • Handling synonyms and different phrasing. • Capturing the "meaning" of a query.	• Excellent at semantic understanding (high recall). • Robust to paraphrasing. • Flexible for various domains.	• Compute-intensive (requires embeddings and vector DB). • Less precise for exact technical terms. • Complex to set up and maintain.
3. Hybrid Search (Dense + Sparse)	• General-purpose production RAG. • Needing both broad context and precise term matching.	• Best overall recall (finds most relevant documents). • Balances strengths of both keyword and semantic search.	• Harder to tune (requires finding the right weights for combination). • Increased system complexity.
4. Reranking (Cross-Encoder)	• Production RAG, after initial retrieval. • When precision in the top results is critical. • If query/document length varies significantly.	• Drastically improves top-K accuracy. • acts as a second, smarter judge.	• High latency (much slower than retrieval). • Cannot find new documents; only improves existing order. • High compute cost.



Grounding & Context Control

Prompt Engineering for Grounding

WEAK PROMPT (Hallucination risk)

System: You are a helpful assistant.

User: What were our Q3 earnings?

Model: (invents numbers from training data)

(No grounding - answers from memory)

STRONG PROMPT (Grounded)

System: You are an expert assistant.

ONLY answer using the [CONTEXT] below.

If unsure, say: I don't know.

[CONTEXT]

Doc A: Q3 revenue grew 23% to \$4.2B.

User: What were our Q3 earnings?

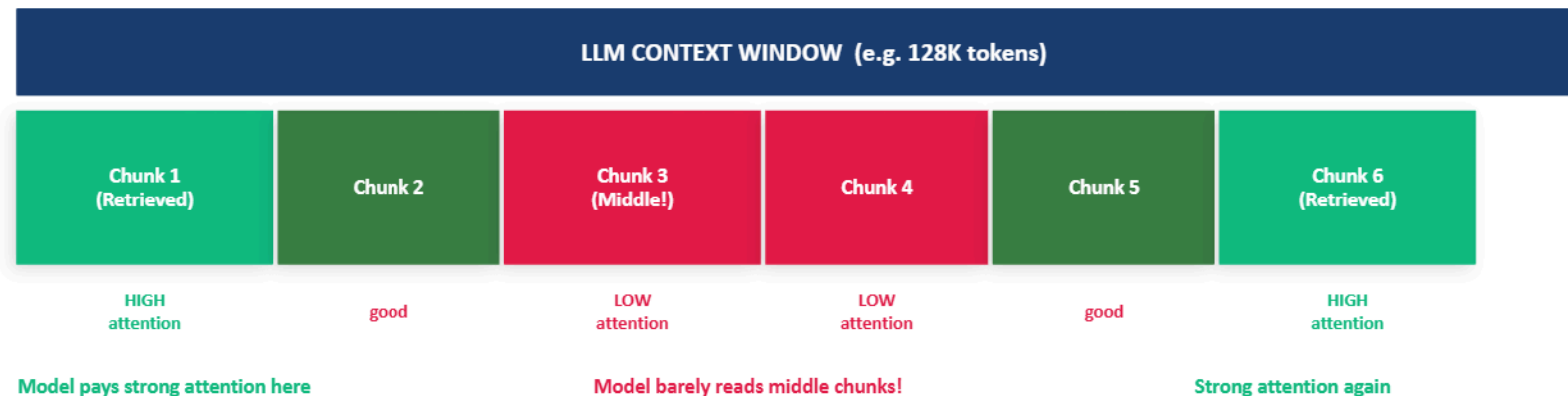
Model: According to Doc A, Q3 revenue

grew 23% to reach \$4.2B.

Key phrases to include in system prompt:

"Answer ONLY using context" · "Say I don't know if unsure" · "Do not use prior knowledge"

The Lost-in-the-Middle Problem



Mitigations:

Reorder by relevance

Put highest-scoring chunks FIRST & LAST in prompt

Limit chunk count

Use 3-5 chunks max, not 20. Quality over quantity.

Context compression

Summarize each chunk before injecting — removes noise

MMR (Max Marginal Relevance)

Diversify retrieved chunks to reduce redundancy

Citation

Attribution: Teaching the Model to Cite

System: "After each claim, add [Source: <doc_name>]"

Answer: Revenue grew 23% in Q3. [Source: annual_report_2024.pdf, p.7]

The cloud segment led growth. [Source: investor_call_Q3.txt, para 3]

Why this matters:

- Users can verify claims instantly
- Auditors can trace every answer back to a document
- Reduces hallucination — model is committed to a source

Citations transform RAG from a "creative writer" into a "precise librarian"

- Eliminating the "Black Box"
- The Hallucination Deterrent
- Auditability & Compliance
- User Confidence

Scoped Search

Metadata Filtering: Narrowing the Search

Filter conditions attached to each chunk at index time:

date >= 2024-01-01

Only search recent documents

department = Sales

Restrict to sales team docs only

language = EN

Avoid cross-language retrieval noise

doc_type = contract

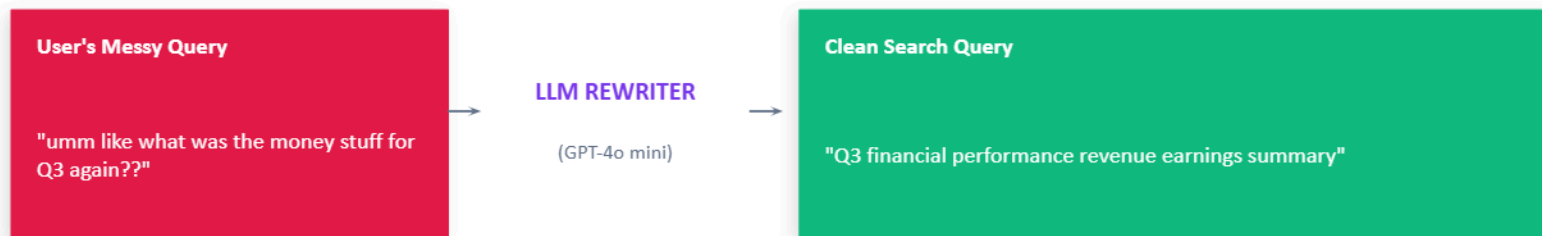
Only search contract-type files

Vector search (semantic search) is great at finding things that sound similar, but it's bad at following strict rules (like "only documents from last month"). Metadata filtering reduces search space → faster retrieval + higher precision



Query Rewriting & Expansion

Bridging Human Questions & Machine Search



Retrieval Quality Impact:

Without Rewriting

Retrieved: general Q3 slide deck (irrelevant)

Missed: Q3 earnings report (exact match)

With Rewriting

Retrieved: Q3 earnings report (top result)

Retrieved: CFO commentary Q3 (bonus hit)

It ensures the retrieval engine has the best possible chance of finding the right documents.

Generate 3-5 Query Variants

Multi-Query: Generate 3-5 Query Variants

Original: How was Q3 financial performance?

Variant 1: Q3 revenue earnings results

Variant 2: third quarter financial summary

Variant 3: Q3 profit loss income statement

Variant 4: quarterly business results July-Sept

Each variant retrieves different docs

Results merged via RRF fusion

Net effect: higher recall, richer context

Instead of sending a single query to the database, we use a LLM as an automated "prompt engineer" to rewrite the original question from multiple perspectives.

- Higher Recall
- Reduced Sensitivity
- Richer Context

Hypothetical Document Embeddings

HyDE: Hypothetical Document Embeddings

- 1 User asks a question
- 2 LLM generates a FAKE answer (hypothetical doc)
- 3 Embed the fake answer (not the query)
- 4 Search: fake-answer vector vs real doc vectors
- 5 Return real docs similar to the fake answer

HyDE works because it transforms the search task from 'find an answer to this question' into '**find a real document that looks like this hypothetical answer.**' By searching for patterns instead of questions, we get much higher precision in our retrieval



Failure Modes in RAG

Retrieval Failure: Not Found

What causes it:

- Chunk too small — no single chunk contains full answer
- Embedding model unfamiliar with domain vocab
- Query vocabulary mismatch with document vocabulary
- Bad chunking — answer split across two chunks

How to detect:

- $\text{Recall@5} < 0.6$ in offline evaluation
- RAG answer says 'I don't know' but doc exists in DB

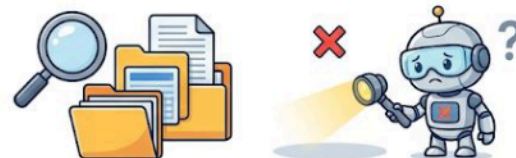
Fixes:

- Use hybrid search (BM25 + dense)
- Fine-tune embedding model on domain data

Failure Modes in RAG

#1 Search Fails

RETRIEVAL FAILURE: NOT FOUND



Vector Database search fails to find relevant information.

Context Overflow: Too Much Noise

What causes it:

- k set too high (k=20+) pulls irrelevant chunks
- Low-quality retrieval, top-20 mostly noise
- Very long chunks eating most of context window
- No reranking — bad docs in the top-k

Symptoms:

- Model ignores the correct chunk (lost-in-middle)
- Model hallucinates despite correct doc being present
- Slow inference (LLM processing too many tokens)

Fixes:

- Reduce k to 3-5 high-quality chunks

Failure Modes in RAG

#2 Context Overwhelms



Hallucination: Model Ignores Context

What causes it:

- Model uses own training knowledge instead of retrieved context
- System prompt too weak — no explicit grounding instruction
- Retrieved context is irrelevant, model falls back to priors
- Temperature too high — model explores beyond context

How to detect:

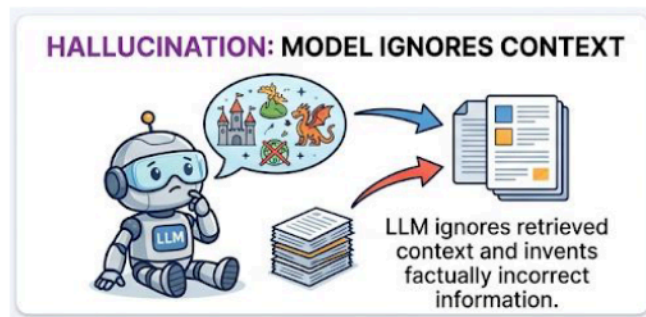
- LLM-as-judge faithfulness score < 0.7
- NLI model: claim not entailed by any retrieved chunk

Fixes:

- Add explicit grounding instruction to system prompt
- Set temperature 0.0–0.3 for factual RAG tasks

Failure Modes in RAG

#3 When the Model Lies



Formatting Errors: Wrong Output Shape

What causes it:

- No output schema defined in the prompt
- No few-shot examples showing expected format
- Model defaults to prose without format instruction
- Complex nested JSON without Pydantic guard

Symptoms:

- JSON parse error crashes downstream pipeline
- Output is plain prose instead of structured list
- Response length far outside specified limits

Fixes:

- Use OpenAI Structured Outputs (`response_format=schema`)

Failure Modes in RAG

#5 Breaks Format

FORMATTING ERRORS: WRONG OUTPUT SHAPE



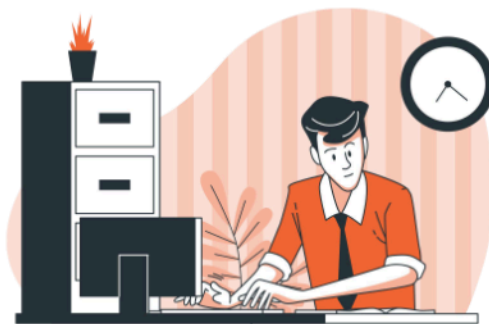
LLM ignores retrieved context and invents factually incorrect information.



Model fails to output data in the required format (e.g., JSON, list).

Guided Hands-on

Open in Google Colab [↗](#)



Assignment



Mini RAG Project

Objective

You will build a Document Q&A Assistant that:

- Loads and chunks your own document
- Indexes it into ChromaDB using Azure embeddings
- Answers questions using a grounded RAG pipeline

Scenario

Build a small but complete RAG system on a document of your choice, following the same architecture demonstrated in the session.

[Open in Google Colab ↗](#)



Thank You